

PP_{Lab4}

Anton Shkumat

May 2026

1 Singleton

1.1 Problem

Mamy obiekt konfiguracyjny klasy `Config`. Nie chcemy za każdym razem generować nowego obiektu z danymi konfiguracyjnymi. Zamiast tego możemy zwracać już raz wygenerowane dane.

Potrzebujemy dokładnie jednej instancji klasy w całym programie, dostępnej globalnie. Zwykła klasa nie gwarantuje, że nie powstanie wiele instancji. Wzorec stosuje się gdy:

- musi istnieć dokładnie jedna instancja klasy,
- instancja musi być dostępna z wielu miejsc w programie,
- chcemy kontrolować dostęp do zasobu współdzielonego.

1.2 Rozwiązanie

Konstruktor jest prywatny. Klasa przechowuje statycznie swoją jedyną instancję. Dostęp odbywa się wyłącznie przez statyczną metodę `getInstance()`, która tworzy instancję przy pierwszym wywołaniu i zwraca tę samą przy każdym kolejnym.

Kluczowe szczegóły implementacyjne

- Prywatne pole statyczne `_instance` jest inicjalnie `None` — instancja tworzona przy pierwszym wywołaniu.
- `getInstance()` sprawdza `if _instance is None` — jeśli tak, tworzy; jeśli nie, zwraca istniejącą.
- W Pythonie Singleton można zrealizować przez `__new__()`, dekorator lub moduł (moduły są z natury singletonami).

1.3 Konsekwencje

- Gwarancja istnienia tylko jednej instancji.
- Globalny punkt dostępu do instancji.

- Narusza zasadę Single Responsibility (klasa zarządza własnym cyklem życia).
- Utrudnia testowanie jednostkowe — trudno podmienić na mock.
- W środowiskach wielowątkowych wymaga synchronizacji.